

TRƯỜNG ĐẠI HỌC SƯ PHẠM KỸ THUẬT HƯNG YÊN
KHOA CÔNG NGHỆ THÔNG TIN

----- oOo -----



BÀI TẬP 02
HỌC MÁY CƠ BẢN

TS. HOÀNG QUỐC VIỆT

Hưng Yên – Tháng 09 năm 2026

BÀI TẬP 02: MẠNG NƠ-RON

A. MỤC TIÊU

Sau bài tập này người học có thể:

- Hiểu được kiến trúc mạng nơ-ron nhân tạo.
- Xây dựng được mạng nơ-ron nhân tạo cho bài toán cụ thể.
- Thực hiện đào tạo và đánh giá mạng nơ-ron đã xây dựng.

B. MÔI TRƯỜNG LÀM VIỆC

Với đặc thù của môn Học máy cơ bản, mục này sẽ liệt kê một số công cụ sử dụng để làm bài tập. Trong bài tập này, người học cần kiểm tra và chắc chắn các phần mềm sau trên máy tính còn hoạt động tốt:

1. Jupyter notebooks.
2. Google colab.

C. TÀI NGUYÊN THAM CHIẾU

Để hoàn thành tốt bài tập này người học nên tham khảo các tài nguyên sau:

STT	Tên tài nguyên	Mô tả tài nguyên
1	diabetes.csv	File dataset.
2	sepsis.csv	File dataset.

D. YÊU CẦU

Bài 1. Xây dựng mạng nơ-ron cho bài toán dự đoán khả năng một bệnh nhân mắc bệnh nhiễm trùng huyết (sepsis) có tử vong hay không? Các yêu cầu (mỗi công việc thực hiện trên 1 cell của Jupyter notebook):

- Yêu cầu 1: Đọc dữ liệu từ file *sepsis.csv*. Chia dữ liệu thành 2 phần: training (80%) và phần còn lại cho testing (20%).
- Yêu cầu 2: Xây dựng mạng nơ-ron có tên *model_sepsis* có 4 lớp ẩn (Mạng nơ-ron này có 6 lớp: 1 Input layer, 4 Hidden layers 8, 16, 32 và 8 nơ-ron, 1 Output layer.
- Yêu cầu 3: Lựa chọn hàm loss và tối ưu hoá cho mô hình đó.

- Yêu cầu 4: Tiến thành training mạng nơ-ron đã xây dựng trên tập dữ liệu đã được xử lý ở bước 1 với 100 epochs và lưu lại các model đã huấn luyện.
- Yêu cầu 5: Đánh giá model đã được training trên tập testing.
- Yêu cầu 6: Huấn luyện lại mô hình 50 epochs và đánh giá lại trên tập testing. Xem kết quả có cải thiện hay không?
- Yêu cầu 7: Huấn luyện lại mô hình 80 epochs và đánh giá lại trên tập testing. Xem kết quả có cải thiện hay không?

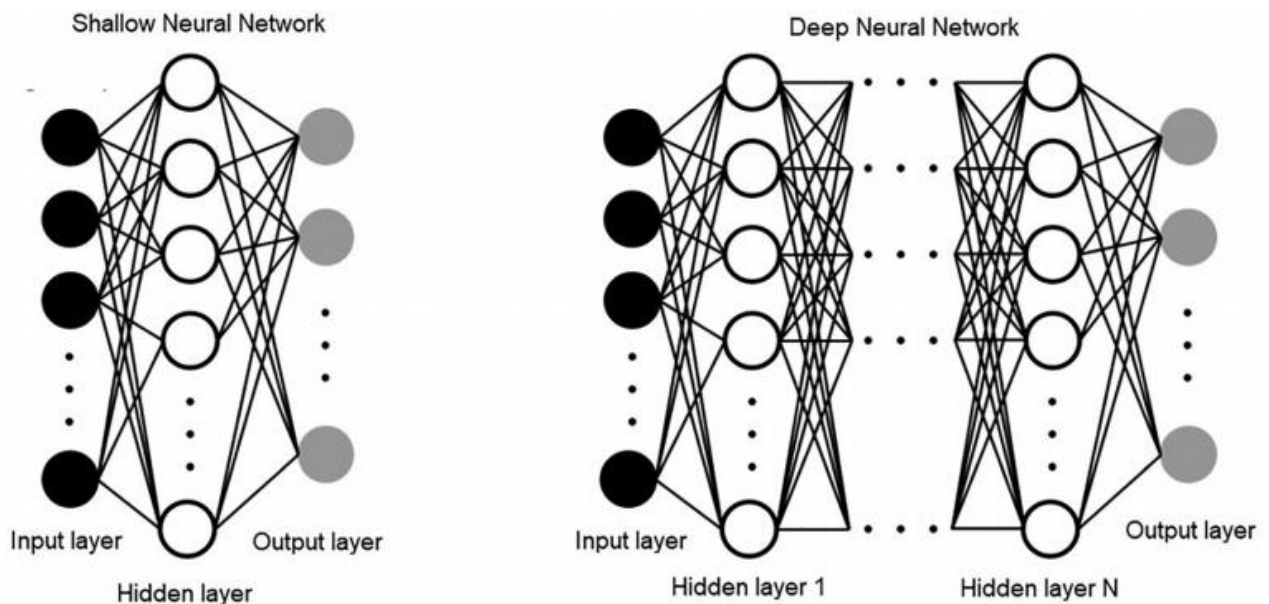
Bài 2. Xây dựng mạng nơ-ron cho bài toán dự đoán khả năng một bệnh nhân có mắc bệnh tiểu đường hay không? (dữ liệu numeric chứa trong csv file). Các yêu cầu (mỗi công việc thực hiện trên 1 cell của Jupyter notebook):

- Yêu cầu 1: Đọc dữ liệu từ file *diabetes.csv*. Chia dữ liệu thành 2 phần: training (85%) và phần còn lại cho testing (15%).
- Yêu cầu 2: Xây dựng mạng nơ-ron:
 - Xây dựng mạng nơ-ron **nông** đặt tên là modelNN1 chỉ có 2 lớp ẩn (Mạng nơ-ron này chỉ có 4 lớp: 1 Input layer, 2 Hidden layer 16 và 32 nơ-ron, 1 Output layer).
 - Xây dựng mạng nơ-ron **sâu** đặt tên là modelNN2 với nhiều lớp ẩn (1 Input layer, nhiều Hidden layers và 1 Output layer).
- Yêu cầu 3: Lựa chọn hàm loss và tối ưu hoá cho cả 2 mô hình.
- Yêu cầu 4: Tiến thành training mạng nơ-ron đã xây dựng trên tập dữ liệu đã được xử lý ở bước 1 với 50 epochs và lưu lại các model đã huấn luyện.
- Yêu cầu 5: Đánh giá model đã được training trên tập testing. So sánh xem mô hình nào tốt hơn: nông hay sâu?
- Yêu cầu 6: Sử dụng *trained model* (lưu ở bước 4) để đánh giá trên tập testing (20%) được xử lý ở bước 1.
- Yêu cầu 7*: Xây dựng ứng dụng thực tế sử dụng trained model.

E. QUY ĐỊNH FILE MÃ NGUỒN

- Người học tạo thư mục *Code* lưu trong máy tính. Sau đó copy các file dataset vào thư mục đó.
- File code bài 1 có định dạng: **Practice02Exercise01_HoVaTen.ipynb**. Ví dụ:

- Practice02Exercise01_NguyenVanA.ipynb.
- Practice02Exercise01_LeAnhB.ipynb...
- File code bài 2 có định dạng: **Practice02Exercise02_HoVaTen.ipynb**.
- Sinh viên cần upload các files kết quả lên link được cung cấp.
- **Kiến trúc mạng nơ-ron:**



- **Code tham khảo:**

- **Bước 1:**

```
# BƯỚC 1 (YÊU CẦU 1): ĐỌC DỮ LIỆU TỪ Sepsis.csv VÀ CHIA DỮ LIỆU LÀM 2 PHẦN: TRAINING (2585) VÀ TESTING (CÒN LẠI)
```

```
"""
```

GIỚI THIỆU DATASET Practice01DatasetSS.CSV: GỒM 7 CỘT QUAN TRỌNG CHỨA THÔNG SỐ VỀ BỆNH NHÂN NHIỄM TRÙNG HUYẾT:

- LOS: Thời gian nằm viện.
- GENDER: Giới tính.
- AGE: Tuổi.
- WEIGHT: Cân nặng.
- GCS TOTAL: Chỉ số GCS.
- TEMPERATURE: Nhiệt độ cơ thể.
- OUTCOME: Tử vong(1)/Sống (0).

6 cột đầu: features, cột cuối cùng (OUTCOME) labels.

```
"""
```

```
import pandas as pd
from tensorflow import keras
from sklearn.model_selection import train_test_split
```

```
# 1.1. Đọc file Practice01DatasetSS.csv sử dụng pandas
columns = ['LOS', 'GENDER', 'AGE', 'WEIGHT', 'GCS TOTAL', 'TEMPERATURE', 'OUTCOME']
df = pd.read_csv('Dataset/sepsis.csv', usecols = columns)

# 1.2. Chia dữ liệu: train (80% = 2585) and test (20% = 647)
X = df[['LOS', 'GENDER', 'AGE', 'WEIGHT', 'GCS TOTAL', 'TEMPERATURE']] # 6 columns

y = df['OUTCOME'] # 1 column
train_size = 2585
X_train, X_test = X.iloc[:train_size], X.iloc[train_size:]
y_train, y_test = y.iloc[:train_size], y.iloc[train_size:]
# X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
# shuffle=True) # Nếu muốn chia ngẫu nhiên

# 1.3. Mã hóa One-hot (One-hot encoding)
y_train_onehot = keras.utils.to_categorical(y_train, 2)
y_test_onehot = keras.utils.to_categorical(y_test, 2)

# Finished
print("Number of Training samples:", len(X_train))
print("Number of Testing samples:", len(X_test))
```

○ **Bước 2:**

```
# BƯỚC 2 (YÊU CẦU 2): XÂY DỰNG MẠNG NƠI RON
from tensorflow.keras.layers import Dense
from tensorflow.keras.models import Sequential

# 2.1. Xây dựng mạng nơ-ron với 3 lớp ẩn
def model_sepsis():
    # Khởi tạo mạng
    model = Sequential()

    # Xây dựng mạng: 1 Input + 3 Hidden + 1 Output
    model.add(Dense(8, input_dim=6, activation='relu'))
    model.add(Dense(16, activation='relu'))
    model.add(Dense(4, activation='relu'))
    model.add(Dense(2, activation='softmax'))

    # Return
    return model

# 2.2 Chọn model (1 lớp ẩn hay nhiều lớp ẩn)
model = model_sepsis()

# 2.4. Hiển thị thông tin model (tùy chọn)
model.summary()
```

○ **Bước 3:**

```
# BƯỚC 3: LỰA CHỌN HÀM MẤT MẮT VÀ TỐI ƯU HOÁ
model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy']) #
Hoặc
# model.compile(loss='categorical_crossentropy',
optimizer=keras.optimizers.Adam(learning_rate=0.0002), metrics=['binary_accuracy'])
```

○ **Bước 4:**

```
# BƯỚC 4: HUẤN LUYỆN (TRAINING MODEL)
from tensorflow import keras
import matplotlib.pyplot as plt
from tensorflow.keras.callbacks import ModelCheckpoint
from tensorflow.keras.layers import Dropout

# 4.1. Training model (có nhiều tùy chọn). Một số tham số:
history = model.fit(X_train, y_train_onehot, validation_split=0.2, epochs=10,
batch_size=16)

# 4.2. Lưu weight và model
# model.save_weights("modelNN_weight.hdf5")
model.save('modelNN_model.h5')

# 4.3. Hiển thị Training and Validation Loss
plt.figure(figsize=(8, 6))
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title('Training and Validation Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
plt.show()
```

○ **Bước 5:**

```
# BƯỚC 5: ĐÁNH GIÁ (TESTING) MÔ HÌNH
import numpy as np
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

# 5.1. Hiển thị kết quả (accuracy) trên tập testing
test_loss, test_acc = model.evaluate(X_test, y_test_onehot)
print('Test loss:', test_loss)
```

```
print('Test accuracy:', test_acc)

# BƯỚC 5 (YÊU CẦU 5): SỬ DỤNG TRAINED MODEL
from tensorflow.keras.models import Sequential, Model, load_model, Sequential

# 5.1. Load lại model
model_loaded = load_model('modelNN_model.h5')
```